

ORION – PROJEKT DOKUMENTÁCIÓ

Ez a dokumentum az Orion rendszer teljes és hivatalos dokumentációja. Tartalmazza a projekt bemutatását, az üzleti logikát, valamint az adatbázis-réteg részletes felépítését (táblák, függvények, triggerek).

Tartalomjegyzék

1. 1. Bevezetés
 1. 1.1. Projekt célja
 2. 1.2. Probléma bemutatása
2. 2. Technológiai Háttér (Supabase / PostgreSQL)
3. 3. Táblák részletes leírása
 1. 3.1. Felhasználók (Users) Modul
 2. 3.2. Cégek (Companies) Modul
 3. 3.3. Álláshirdetések és Munkaviszony
 4. 3.4. Kommunikáció és Értesítések
4. 4. Táblakapcsolatok (ER összefoglaló)
5. 5. Adatbázis Függvények (RPC)
6. 6. Triggerek
7. 7. Backend Szerver Architektúra (src/server)
 1. 7.1. Middleware és Realtime
 2. 7.2. API Végpontok és Szolgáltatások (Routes & Services)

1. Bevezetés

1.1. Projekt célja

Az Orion egy modern diákmunka megosztó és közvetítő platform – hasonlóan a hazai piacon ismert profession.hu-hoz, azonban kifejezetten a diákmunkára és a diákok számára optimalizált változatban.

A projekt legfőbb küldetése, hogy egy transzparens, gyors és közvetlen kapcsolatot biztosítson a munkát kereső diákok és az őket foglalkoztatni kívánó cégek között, kihagyva a hagyományos, lassú és gyakran a diákok számára anyagilag hátrányos harmadik feleket (például közvetítő diákszövetkezeteket).

1.2. Probléma bemutatása és az Orion megoldása

A jelenlegi diákmunka-piacon a diákok legtöbbször diákszövetkezeteken keresztül helyezkednek el és dolgoznak. Ez a rendszer gyakran kényelmetlen, adminisztrációval és bürokráciával terhelt a bonyolult szerződéskötési folyamatok miatt. Továbbá a közvetítők (szövetkezetek) beiktatása miatt a diákok sok esetben jelentősen alacsonyabb órabért kapnak kézhez ahhoz képest, amit a cégek ténylegesen kifizetnek értük, de sokszor inkorrekt elszámolások és rejtett levonások – népiesen szólva "átverések" – is előfordulnak.

A megoldás: Az Orion rendszerében a diákok és a munkáltató cégek *közvetlenül* egymással szerződnek és tartják a kapcsolatot. Nem az Orion (mint platform) szerződik velük közvetítőként, hanem mi csupán a piacteret és a biztonságos kommunikációs, illetve adminisztrációs szoftvert biztosítjuk számukra. Ezzel kiküszöböljük a közvetítő hálózatok és szövetkezetek minden hátrányát: jelentősen lecsökken a szerződéskötési, adminisztrációs teher, és ami a legfontosabb, a diákok nem lesznek anyagilag megkárosítva. A cégek pedig egy modern önkiszolgáló felületen oszthatják meg az álláshirdetéseiket, közvetlenül érhetik el a jelentkező diákokat, teljesen maguk dönthetnek a felvételükről, a feladatok kiosztásáról, vagy adott esetben a munkaviszonyuk megszüntetéséről is.

2. Technológiai Háttér

Miért a Supabase / PostgreSQL?

A projekt adatbázis-motorjaként és backend szolgáltatásaként a **Supabase** platformot használjuk, amely egy nyílt forráskódú PostgreSQL-alapú megoldás. Főbb indokok:

- **PostgreSQL Alapok** Robusztus, relációs adatbázis; kiváló teljesítmény, megbízhatóság és tranzakciókezelés.
- **Biztonság és Titkosítás** Beépített pgcrypto bővítmény: a szenzitív adatok (személyi igazolvány, lakcímkártya) adatbázis-szintű szimmetrikus titkosítással (pgp_sym_encrypt) tárolódnak.

- **Gyors Backend Fejlesztés** Automatikus REST és GraphQL API generálás, valós idejű (real-time) feliratkozások – ez utóbbi elengedhetetlen a beépített chat funkcióhoz.
- **Beépített Auth és RLS** Row Level Security: a jogosultságkezelés közvetlenül az adatbázis rétegben valósul meg.
- **PL/pgSQL RPC** Az üzleti logika egy része tárolt eljárásokban él, minimalizálva a hálózati round-trip-ek számát.

Kiterjesztés: `CREATE EXTENSION IF NOT EXISTS pgcrypto;` — a titkosított dokumentumok tárolásához szükséges kiterjesztés. Minden Supabase projekten aktiválni kell.

3. Táblák részletes leírása

Az adatbázis négy fő logikai modulra bontható: **Felhasználók**, **Cégek**, **Álláshirdetések / Munkaviszony** és **Kommunikáció / Üzenetek**.

3.1 Felhasználók (Users) Modul

users

A munkavállalók (álláskereső) összes alapadatát tároló fő tábla.

Mező	Típus	Constraint	Leírás
<code>id</code> PK	SERIAL	PRIMARY KEY	Automatikusan generált egyedi azonosító
<code>email</code>	TEXT	UNIQUE NOT NULL	Egyedi email cím, belépéshez és értesítésekhez
<code>lname</code>	TEXT	NOT NULL	Felhasználó vezetékneme
<code>fname</code>	TEXT	NOT NULL	Felhasználó keresztneme
<code>phone_number</code>	TEXT	NOT NULL	Telefonszám
<code>birth_place</code>	TEXT	NOT NULL	Születési hely
<code>birth_date</code>	DATE	NOT NULL	Születési dátum

address	TEXT	NOT NULL	Lakcím
tax_number	TEXT	NOT NULL	Adóazonosító szám
nationality	TEXT	NOT NULL	Állampolgárság
terms_accepted	BOOLEAN	DEFAULT FALSE NOT NULL	ÁSZF elfogadásának jelzése
activated	BOOLEAN	DEFAULT FALSE NOT NULL	Fiók aktiváltsági állapot (email megerősítés után TRUE)
short_bio	TEXT	NOT NULL	Rövid bemutatkozás / önéletrajzi szöveg
qualifications	TEXT	NOT NULL	Végzettségek, képesítések
join_date	TIMESTAMP	DEFAULT NOW() NOT NULL	Regisztráció időpontja

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  lname TEXT NOT NULL,
  fname TEXT NOT NULL,
  phone_number TEXT NOT NULL,
  birth_place TEXT NOT NULL,
  birth_date DATE NOT NULL,
  address TEXT NOT NULL,
  tax_number TEXT NOT NULL,
  nationality TEXT NOT NULL,
  terms_accepted BOOLEAN DEFAULT FALSE NOT NULL,
  activated BOOLEAN DEFAULT FALSE NOT NULL,
  short_bio TEXT NOT NULL,
  qualifications TEXT NOT NULL,
  join_date TIMESTAMP DEFAULT NOW() NOT NULL
);
```

user_credentials

A felhasználói jelszavak biztonságos, hashelt tárolása. Az alaptáblától elkülönítve, elv: minimális jogosultság elve alapján csak erre a táblára vonatkozó olvasási jog szükséges az autentikációhoz.

Mező	Típus	Constraint	Leírás
------	-------	------------	--------

<code>user_id</code> PK FK	INTEGER	REFERENCES users(id) ON DELETE CASCADE	Hivatkozás a users táblára
<code>password_hash</code>	TEXT	NOT NULL	Hashelt jelszó (pl. bcrypt)

```
CREATE TABLE user_credentials (
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  password_hash TEXT NOT NULL,
  PRIMARY KEY (user_id)
);
```

user_documents

Személyi okmányok másolatainak **titkosított** tárolója. Az adatokat pgp_sym_encrypt segítségével szimmetrikusan titkosítva tároljuk, nyers szöveg soha nem kerül az adatbázisba.

⚠ Szenzitív adat: A `personal_id` és `address_card_number` mezők BYTEA típusban titkosított ciphertext-et tárolnak. Közvetlen lekérdezés értelmetlen – kizárólag a `get_decrypted_documents()` RPC függvényen át érhetők el.

Mező	Típus	Constraint	Leírás
<code>user_id</code> PK FK	INTEGER	REFERENCES users(id) ON DELETE CASCADE	Hivatkozás a users táblára
<code>personal_id</code> ENC	BYTEA	NOT NULL	Személyi igazolvány száma – titkosítva
<code>address_card_number</code> ENC	BYTEA	NOT NULL	Lakcímkártya száma – titkosítva

```
CREATE TABLE user_documents (
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
  personal_id BYTEA NOT NULL,
  address_card_number BYTEA NOT NULL,
  PRIMARY KEY (user_id)
);
```

user_email_tokens

Regisztrációs email-megerősítési és jelszó-helyreállítási tokenek tárolása. A `expires_at` mező biztosítja, hogy a tokenek korlátozott ideig érvényesek.

Mező	Típus	Constraint	Leírás
<code>user_id</code> PK FK	INTEGER	REFERENCES users(id) ON DELETE CASCADE	Hivatkozás a users táblára
<code>token</code>	TEXT	UNIQUE NOT NULL	Egyedi, véletlenszerű token string
<code>expires_at</code>	TIMESTAMP	NOT NULL	Token lejáratának időpontja

```
CREATE TABLE user_email_tokens (
  user_id INTEGER PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  token TEXT UNIQUE NOT NULL,
  expires_at TIMESTAMP NOT NULL
);
```

user_statistics

A felhasználói profil látogatási és önéletrajz-megtekintési statisztikái. Automatikusan kerül létrehozásra a `trg_create_user_statistics` trigger által, új felhasználó regisztrálásakor.

Mező	Típus	Constraint	Leírás
<code>user_id</code> PK FK	INTEGER	REFERENCES users(id) ON DELETE CASCADE	Hivatkozás a users táblára
<code>profile_views</code>	INTEGER	NOT NULL DEFAULT 0	Profiloldal megtekintéseinek száma
<code>resume_views</code>	INTEGER	NOT NULL DEFAULT 0	Önéletrajz megtekintéseinek száma

```
CREATE TABLE user_statistics (
  user_id INTEGER PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  profile_views INTEGER NOT NULL DEFAULT 0,
  resume_views INTEGER NOT NULL DEFAULT 0
);
```

favorites

A felhasználók által elmentett / kedvelt álláshirdetések nyilvántartása. Összetett elsődleges kulcs biztosítja, hogy egy felhasználó egy hirdetést csak egyszer menthet el.

Mező	Típus	Constraint	Leírás
user_id PK FK	INTEGER	REFERENCES users(id) ON DELETE CASCADE	Hivatkozás a users táblára
advertisement_id PK FK	INTEGER	REFERENCES advertisement(id) ON DELETE CASCADE	Hivatkozás az advertisement táblára

```
CREATE TABLE favorites (
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  advertisement_id INTEGER NOT NULL REFERENCES advertisement(id) ON DELETE CASCADE,
  PRIMARY KEY (user_id, advertisement_id)
);
```

3.2 Cégek (Companies) Modul

companies

A munkáltatóként regisztráló cégek összes alapadatát tároló fő tábla.

Mező	Típus	Constraint	Leírás
id PK	SERIAL	PRIMARY KEY	Automatikusan generált egyedi azonosító
email	TEXT	UNIQUE NOT NULL	Egyedi email cím
name	TEXT	NOT NULL	Cég neve
address	TEXT	NOT NULL	Székhely / cím
tax_number	TEXT	NOT NULL	Adószám
contact_person_name	TEXT	NOT NULL	Kapcsolattartó személy neve
activity_scope	TEXT	NOT NULL	Tevékenységi kör / iparág
website	TEXT	–	Cég weboldalának URL-je (opcionális)
short_description	TEXT	NOT NULL	Rövid bemutatkozás / cégismertető

terms_accepted	BOOLEAN	DEFAULT FALSE NOT NULL	ÁSZF elfogadásának jelzése
verified	BOOLEAN	DEFAULT FALSE NOT NULL	Admin által manuálisan hitelesített cég
join_date	TIMESTAMP	DEFAULT NOW() NOT NULL	Regisztráció időpontja
phone_number	TEXT	NOT NULL	Telefonszám

```
CREATE TABLE companies (
  id SERIAL PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  address TEXT NOT NULL,
  tax_number TEXT NOT NULL,
  contact_person_name TEXT NOT NULL,
  activity_scope TEXT NOT NULL,
  website TEXT,
  short_description TEXT NOT NULL,
  terms_accepted BOOLEAN DEFAULT FALSE NOT NULL,
  verified BOOLEAN DEFAULT FALSE NOT NULL,
  join_date TIMESTAMP DEFAULT NOW() NOT NULL,
  phone_number TEXT NOT NULL
);
```

company_credentials

A cégek belépési jelszavainak hashelt tárolása. Felépítése megegyezik a user_credentials táblával.

Mező	Típus	Constraint	Leírás
company_id PK FK	INTEGER	REFERENCES companies(id) ON DELETE CASCADE	Hivatkozás a companies táblára
password_hash	TEXT	NOT NULL	Hashelt jelszó

```
CREATE TABLE company_credentials (
  company_id INTEGER REFERENCES companies(id) ON DELETE CASCADE,
  password_hash TEXT NOT NULL,
  PRIMARY KEY (company_id)
);
```

company_email_tokens

A cégek email-megerősítési folyamataihoz tartozó tokenek. Megegyező struktúrájú a `user_email_tokens` táblával.

Mező	Típus	Constraint	Leírás
<code>company_id</code> PK FK	INTEGER	REFERENCES companies(id) ON DELETE CASCADE	Hivatkozás a companies táblára
<code>token</code>	TEXT	UNIQUE NOT NULL	Egyedi token string
<code>expires_at</code>	TIMESTAMP	NOT NULL	Token lejáratának időpontja

```
CREATE TABLE company_email_tokens (  
  company_id INTEGER PRIMARY KEY REFERENCES companies(id) ON DELETE CASCADE,  
  token TEXT UNIQUE NOT NULL,  
  expires_at TIMESTAMP NOT NULL  
);
```

3.3 Álláshirdetések és Munkaviszony (Ads & Jobs)

advertisement

A cégek által meghirdetett pozíciók adatai. A `click_count` mező az `increment_click_count()` RPC függvényen át kerül növelésre, valahányszor egy felhasználó megtekinti a hirdetést.

Mező	Típus	Constraint	Leírás
<code>id</code> PK	SERIAL	PRIMARY KEY	Egyedi azonosító
<code>position</code>	TEXT	NOT NULL	Munkakör megnevezése
<code>hourly_wage</code>	INT	NOT NULL	Órabér (Ft)
<code>tasks</code>	TEXT	–	Elvégzendő feladatok leírása
<code>requirements</code>	TEXT	–	Elvárások a jelöltekkel szemben
<code>job_description</code>	TEXT	–	Részletes munkaköri leírás

is_active	BOOLEAN	DEFAULT TRUE	Aktív-e a hirdetés?
click_count	INT	DEFAULT 0	Hirdetés megtekintéseinek száma
search_start	TIMESTAMP	DEFAULT NOW()	Keresés megkezdésének időpontja
company_id FK	INTEGER	REFERENCES companies(id) ON DELETE CASCADE	Hirdető cég azonosítója

```
CREATE TABLE advertisement (
  id SERIAL PRIMARY KEY,
  position TEXT NOT NULL,
  hourly_wage INT NOT NULL,
  tasks TEXT,
  requirements TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  click_count INT DEFAULT 0,
  search_start TIMESTAMP DEFAULT NOW(),
  company_id INTEGER REFERENCES companies(id) ON DELETE CASCADE,
  job_description TEXT
);
```

job_applications

A munkavállalók álláshirdetésekre leadott jelentkezéseit tárolja, és nyomon követi azok státuszát (submitted → accepted / rejected).

Mező	Típus	Constraint	Leírás
id PK	SERIAL	PRIMARY KEY	Egyedi azonosító
user_id FK	INTEGER	NOT NULL REFERENCES users(id) ON DELETE CASCADE	Jelentkező felhasználó
advertisement_id FK	INTEGER	NOT NULL REFERENCES advertisement(id) ON DELETE CASCADE	Érintett álláshirdetés
status	TEXT	NOT NULL DEFAULT 'submitted'	Státusz: submitted / accepted / rejected

`last_updated`

TIMESTAMP

NOT NULL DEFAULT NOW()

Utolsó
státuszváltozás
időpontja

```
CREATE TABLE job_applications (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  advertisement_id INTEGER NOT NULL REFERENCES advertisement(id) ON DELETE CASCADE,  
  status TEXT NOT NULL DEFAULT 'submitted',  
  last_updated TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

employees

Sikeres felvétel után a munkaviszony formalizált nyilvántartása. Ide kerül az összes aktív és lezárt alkalmazotti kapcsolat.

Mező	Típus	Constraint	Leírás
<code>id</code> PK	SERIAL	PRIMARY KEY	Egyedi azonosító
<code>user_id</code> FK	INTEGER	NOT NULL REFERENCES users(id) ON DELETE CASCADE	Alkalmazott felhasználó
<code>company_id</code> FK	INTEGER	NOT NULL REFERENCES companies(id) ON DELETE CASCADE	Munkáltató cég
<code>position</code>	TEXT	NOT NULL	Pozíció / munkakör kód
<code>job_title</code>	TEXT	NOT NULL	Megjelenített állásTitle
<code>hourly_wage</code>	INT	NOT NULL	Megállapodott órabér (Ft)
<code>hire_date</code>	TIMESTAMP	NOT NULL DEFAULT NOW()	Felvétel dátuma

```
CREATE TABLE employees (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  company_id INTEGER NOT NULL REFERENCES companies(id) ON DELETE CASCADE,  
  position TEXT NOT NULL,  
  job_title TEXT NOT NULL,  
  hourly_wage INT NOT NULL,
```

```
hire_date TIMESTAMP NOT NULL DEFAULT NOW()
);
```

3.4 Kommunikáció és Értesítések

messages

Kétoldalú, valós idejű privát chat üzenetek tárolása Felhasználó–Cég viszonylatban. A sender_type CHECK constraint garantálja, hogy csak érvényes küldőtípus kerülhessen az adatbázisba.

Mező	Típus	Constraint	Leírás
id PK	SERIAL	PRIMARY KEY	Egyedi üzenet-azonosító
company_id FK	INTEGER	NOT NULL REFERENCES companies(id)	Érintett cég
user_id FK	INTEGER	NOT NULL REFERENCES users(id)	Érintett felhasználó
sender_type	TEXT	NOT NULL CHECK IN ('USER', 'COMPANY')	Ki küldte az üzenetet
message	TEXT	NOT NULL	Az üzenet szöveges tartalma
is_read	BOOLEAN	NOT NULL DEFAULT FALSE	Olvasottsági állapot
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Küldés időpontja

```
CREATE TABLE messages (
  id SERIAL PRIMARY KEY,
  company_id INTEGER NOT NULL REFERENCES companies(id),
  user_id INTEGER NOT NULL REFERENCES users(id),
  sender_type TEXT NOT NULL CHECK (sender_type IN ('USER', 'COMPANY')),
  message TEXT NOT NULL,
  is_read BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

system_messages

Adminisztrátorok által küldött rendszerszintű értesítések (pl. karbantartási figyelmeztetések, globális hírek). A target_type és target_id kombinációja lehetővé teszi célzott (egy adott felhasználó/cég) vagy globális (NULL target_id) küldést.

Mező	Típus	Constraint	Leírás
id PK	SERIAL	PRIMARY KEY	Egyedi azonosító
title	VARCHAR(255)	NOT NULL	Üzenet tárgya / címe
message	TEXT	NOT NULL	Üzenet törzse
target_type	TEXT	NOT NULL	Célpont típusa: pl. 'USER', 'COMPANY', 'ALL'
target_id	INT	NULL	Konkrét célpont ID-ja (NULL = mindenki)
created_at	TIMESTAMP	NOT NULL DEFAULT NOW()	Létrehozás időpontja
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Aktív-e az üzenet?

system_message_reads

Az egyes rendszerüzenetek olvasottságának nyilvántartása célpontonként.

Mező	Típus	Constraint	Leírás
id PK	SERIAL	PRIMARY KEY	Egyedi azonosító
message_id FK	INT	NOT NULL REFERENCES system_messages(id)	Melyik üzenet
target_type	TEXT	NOT NULL	Ki olvasta (típus)
target_id	INT	NULL	Ki olvasta (ID)

```
CREATE TABLE system_messages (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,
```

```

    message TEXT NOT NULL,
    target_type TEXT NOT NULL,
    target_id INT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE system_message_reads (
    id SERIAL PRIMARY KEY,
    message_id INT NOT NULL,
    target_type TEXT NOT NULL,
    target_id INT NULL,
    FOREIGN KEY (message_id) REFERENCES system_messages(id)
);

```

4. Táblakapcsolatok (ER összefoglaló)

Az alábbi táblázat az összes idegen kulcs kapcsolatot összefoglalja. Minden ON DELETE CASCADE kapcsolat garantálja, hogy egy szülő entitás törlése esetén az összes hozzá tartozó gyermek sor automatikusan törlődik, megőrizve az adatbázis konzisztenciáját.

Gyermek tábla	Külső kulcs mező	→ Szülő tábla	Szülő kulcs	Viselkedés törléskor
user_credentials	user_id	users	id	CASCADE
user_documents	user_id	users	id	CASCADE
user_email_tokens	user_id	users	id	CASCADE
user_statistics	user_id	users	id	CASCADE
favorites	user_id	users	id	CASCADE
favorites	advertisement_id	advertisement	id	CASCADE
company_credentials	company_id	companies	id	CASCADE
company_email_tokens	company_id	companies	id	CASCADE
advertisement	company_id	companies	id	CASCADE
job_applications	user_id	users	id	CASCADE

job_applications	advertisement_id	advertisement	id	CASCADE
employees	user_id	users	id	CASCADE
employees	company_id	companies	id	CASCADE
messages	company_id	companies	id	–
messages	user_id	users	id	–
system_message_reads	message_id	system_messages	id	–

5. Adatbázis Függvények (RPC)

Az üzleti logika egy része közvetlenül PL/pgSQL tárolt eljárásokban él. Ezeket Supabase RPC hívásokon keresztül lehet meghívni a kliensekből. Minden kritikus függvény `SECURITY DEFINER` módban fut a megfelelő jogosultság-kezelés érdekében.

Regisztrációs függvények

`register_user_v1(...) → JSONB`

Tranzakcionális felhasználói regisztráció. Egyetlen hívással, atomi módon elvégzi:

1. User sor beszúrása a `users` táblába
2. Jelszó-hash mentése a `user_credentials` táblába
3. Személyi okmányok titkosítva tárolása (`insert_encrypted_documents` hívásával)

Hiba esetén az egész tranzakció visszagörget (`EXCEPTION WHEN OTHERS`). Visszatérési érték: `{"success": true, "user_id": <id>}`.

Paraméter	Típus	Leírás
p_email	TEXT	Email cím
p_password_hash	TEXT	Előre hashelt jelszó

p_lname	TEXT	Vezetéknév
p_fname	TEXT	Keresztnév
p_phone_number	TEXT	Telefonszám
p_birth_place	TEXT	Születési hely
p_birth_date	DATE	Születési dátum
p_address	TEXT	Lakcím
p_nationality	TEXT	Állampolgárság
p_short_bio	TEXT	Rövid bemutatkozás
p_qualifications	TEXT	Végzettségek
p_tax_number	TEXT	Adóazonosító
p_terms_accepted	BOOLEAN	ÁSZF elfogadva?
p_personal_id	TEXT	Személyi igazolvány száma (titkosítandó)
p_address_card_number	TEXT	Lakcímkártya száma (titkosítandó)
p_encryption_key	TEXT	Titkosítási kulcs

register_company_v1(...) → JSONB

Tranzakcionális céges regisztráció. Atomi módon elvégzi:

1. Cég sor beszúrása a `companies` táblába
2. Jelszó-hash mentése a `company_credentials` táblába

Visszatérési érték: `{"success": true, "company_id": <id>}`.

Paraméter	Típus	Leírás
p_email	TEXT	Email cím

p_password_hash	TEXT	Hashelt jelszó
p_name	TEXT	Cég neve
p_address	TEXT	Székhely
p_tax_number	TEXT	Adószám
p_contact_person_name	TEXT	Kapcsolattartó neve
p_activity_scope	TEXT	Tevékenységi kör
p_website	TEXT	Weboldal (opcionális)
p_short_description	TEXT	Cégismertető
p_terms_accepted	BOOLEAN	ÁSZF elfogadva?
p_phone_number	TEXT	Telefonszám

Dokumentum-titkosítási függvények

`insert_encrypted_documents(p_user_id, p_personal_id, p_address_card_number, p_encryption_key) → void`

Személyi okmányok első mentése. A `pgp_sym_encrypt` segítségével titkosítja a megadott szövegeket és `BYTEA` formátumban elmenti a `user_documents` táblába. Hívja a `register_user_v1`.

`update_encrypted_documents(p_user_id, p_personal_id, p_address_card_number, p_encryption_key) → void`

Meglévő titkosított dokumentumok frissítése. Felülírja a korábban eltárolt értékeket az újabb, szintén titkosított adatokkal.

`get_decrypted_documents(p_user_id, p_encryption_key) → TABLE(personal_id text, address_card_number text)`

A titkosított dokumentumok visszafejtése és visszaadása. A `pgp_sym_decrypt` segítségével oldja fel a titkosítást, és visszaadja az olvasható szöveget. Kizárólag megfelelő `encryption key` megadása esetén működik.

```
-- Példa hívás:  
SELECT * FROM get_decrypted_documents(42, 'titkos_kulcs');
```

Statisztika és Dashboard függvények

`get_company_dashboard_stats(target_company_id int) → JSON`

Egy cég főbb KPI-jait adja vissza egyetlen adatbázis-hívásban, minimalizálva a hálózati lekérdezések számát. Visszaadott mezők:

- `views` – hirdetések összes kattintásszáma (`click_count` összege)
- `applicants` – összes beérkezett jelentkezés száma
- `employees` – aktív alkalmazottak száma
- `conversion` – konverziós arány: $(\text{applicants} / \text{views}) \times 100 \%$ -ban

`get_user_dashboard_stats(p_user_id int) → JSON`

Egy felhasználó dashboard statisztikáit adja vissza. Visszaadott mezők:

- `total_applications` – összes leadott jelentkezés
- `profile_views` – profil megtekintések száma
- `resume_views` – önéletrajz megtekintések száma
- `accepted_applications` – elfogadott jelentkezések száma

`increment_click_count(ad_id int) → void`

Az adott hirdetés `click_count` számlálóját növeli eggyel. Meghívandó minden alkalommal, amikor egy felhasználó megnyit egy álláshirdetést.

`increment_profile_views(p_user_id int) → void`

A `user_statistics` tábla `profile_views` mezőjét növeli eggyel a megadott felhasználó esetén.

`increment_resume_views(p_user_id int) → void`

A `user_statistics` tábla `resume_views` mezőjét növeli eggyel a megadott felhasználó esetén.

Chat / Üzenetküldés függvények

`get_user_chat_partners(p_user_id int) → TABLE`

Visszaadja a felhasználóval chat-kapcsolatban álló összes cég listáját, cégenkénti bontásban a **legutóbbi üzenettel**. Eredmény oszlopai: `company_id`, `company_name`, `message_id`, `sender_type`, `message`, `is_read`, `created_at`.

`get_company_chat_partners(p_company_id int) → TABLE`

Visszaadja a céggel chat-kapcsolatban álló összes felhasználó listáját, utolsó üzenet időpontja szerint csökkenő sorrendben. Eredmény oszlopai: `user_id`, `user_lname`, `user_fname`, `last_message_at`.

6. Triggererek

`trg_create_user_statistics`

Automatikusan lefut minden új felhasználó regisztrációja után (AFTER INSERT ON users). Létrehozza a hozzá tartozó `user_statistics` sort alapértékekkel (`profile_views = 0`, `resume_views = 0`), így biztosítva, hogy minden felhasználóhoz mindig létezik statisztika sor.

```
CREATE OR REPLACE FUNCTION create_user_statistics()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO user_statistics (user_id)
    VALUES (NEW.id);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_create_user_statistics
AFTER INSERT ON users
FOR EACH ROW
EXECUTE FUNCTION create_user_statistics();
```

7. Backend Szerver Architektúra (src/server)

A szerver Node.js és Express.js alapokon fut, Typescript nyelven írva. A fő belépési pont a `server.ts`, ahol az Express alkalmazás beállítása (CORS, Cookie-parser), a végpontok mountolása és a WebSocket szerver inicializálása történik. A port alapértelmezetten a 4000.

7.1. Middleware és Realtime

auth.ts (Middleware)

Összefoglalás: A kliensek (felhasználók és cégek) autentikációját és jogosultság-ellenőrzését végző biztonsági réteg.

- **Folyamat:**
 1. A `verifyToken` kiolvassa a HTTP-only `auth_token` sütiből a JSON Web Token-t.
 2. Érvényesíti a JWT aláírást, majd a kinyert adatokat (`userId`, `companyId`, `userType`) az Express Request objektumába csatolja.
 3. A `verifyUser` és `verifyCompany` ellenőrzi, hogy a tokenből kinyert azonosító valóban létezik-e az adatbázisban, megakadályozva a törölt fiókkal való hozzáférést.

RealtimeServer.ts (WebSocket)

Összefoglalás: Valós idejű WebSocket szerver a chat üzenetek azonnali kézbesítésére.

- **Folyamat:**
 1. `ws` könyvtár segítségével WebSocket szervert hoz létre.
 2. Feliratkozik a Supabase `postgres_changes` csatornán a `messages` tábla új (INSERT) eseményeire.
 3. Új adatbázis bejegyzés esetén a payloadot rápumpálja az összes aktív WebSocket kliensnek (Broadcast).

7.2. API Végpontok és Szolgáltatások (Routes & Services)

Auth Modul (authRoute.ts / authService.ts)

A felhasználók és cégek bejelentkezését, regisztrációját és token ellenőrzését végzi a rendszer.

Végpont	Függvény	Összefoglalás
POST /Api/auth/user/login	<code>loginUser</code>	Felhasználói bejelentkezést kezel, JWT sütit állít be.

POST /Api/auth/user/register	registerUser	Új felhasználót regisztrál, elmentve a titkosított adatokat.
POST /Api/auth/company/login	loginCompany	Céges bejelentkezést kezel, JWT sütit állít be.
POST /Api/auth/company/register	registerCompany	Új munkáltatót regisztrál az adatbázisba.
POST /Api/auth/logout	–	Törli az 'auth_token' cookie-t, kijelentkeztetve a felhasználót.
GET /Api/auth/status	verifyToken	Ellenőrzi a hitelesítési státuszt és visszaadja a profil típusát.

- **Felhasználói Login folyamat:**

1. Megadott adatok validálása Zod-dal. 2. A users táblából email alapján a hashelt jelszó lekérése. 3. bcrypt.compare használatával jelszó egyezés ellenőrzése. 4. Fiók aktivációs státusz vizsgálata. 5. JWT token (auth_token) generálása és beállítása kliens sütiként.

- **Céges Login folyamat:**

1. Ugyanaz a validáció és bcrypt ellenőrzés, de a companies táblán. 2. A generált JWT tokenben a userType: 'company' érték szerepel.

- **Felhasználó Regisztrációs folyamat:**

1. Bemeneti validáció. 2. Jelszó hashelése bcrypt segítségével. 3. A register_user_v1 RPC meghívása, mely végrehajtja a titkosítást és adatbázis beillesztést egy tranzakcióban.

- **Cég Regisztrációs folyamat:**

1. Bemeneti validáció. 2. Jelszó hashelése bcrypt-tel. 3. A register_company_v1 tranzakcionális RPC futtatása a biztos adatkezelés érdekében.

- **Logout (kijelentkezés) folyamat:**

1. A clearCookie('auth_token') hívással a szerver törli a böngésző hitelesítő sütijét.

- **Verify Token (státusz ellenőrzés) folyamat:**

1. A kérés sütijéből kinyeri a JWT-t. 2. Érvényesíti azt. 3. Visszaadja a dekódolt token (benne a userType-al és ID-val), hogy a kliens tudja, érvényes-e a bejelentkezése.

Felhasználó Modul (userRoute.ts / userService.ts)

A diákok profil-alapadatait, statisztikáit, CV-jeit (önéletrajz) és elmentett hirdetéseit (kedvencek) kezeli.

Végpont	Függvény	Összefoglalás
GET /Api/users/me	getUserProfile	Visszaadja a felhasználó alapadatokat, visszafejti az okmányokat és jelzi, ha van feltöltve CV.
PATCH /Api/users/me	updateUserProfile	Frissíti a felhasználói profilt és újra-titkosítja a frissített okmányokat.
POST /Api/users/me/resume	uploadResume	Egy darab (max 5MB) önéletrajz fájlt tölt fel a Supabase Storage-be.
DELETE /Api/users/me/resume	deleteResume	Véglegesen törli a felhasználó önéletrajzát a tárolóból.
GET /Api/users/me/stats	getUserDashboardData	Egységesített dashboard statisztikákat (RPC-vel jelentkezések, nézettség) kér le.
GET /Api/users/me/resume	getResumeUrl	Ideiglenes letöltési linket generál a tárolt CV-hez.
POST /Api/users/me/favorites	addFavorite	Felvesz egy álláshirdetést a kedvencek listájába.
GET /Api/users/me/favorites	getFavorites	Lekéri a kedvelt hirdetések ID-jait vagy teljes listáját.
DELETE /Api/users/me/favorites/:id	removeFavorite	Eltávolít egy hirdetést a kedvencekből.

POST /Api/users/:id/views

incrementProfileViews

Megtekintéskor megnöveli a profil látogatottsági mutatóját (RPC).

- **Profil lekérés folyamat:**

1. Adatbázis keresés a users táblában UserId alapján. 2. A get_decrypted_documents RPC meghívásával a személyes adatok (titkosítás) biztonságos visszafejtése. 3. Feltöltött CV meglétének ellenőrzése a Storageból.

- **Profil frissítés folyamat:**

1. A validált bejövő adatok (név, cím, sul) alapján a users tábla felülírása és egyidejűleg az encrypting RPC lefutása az új adatokkal.

- **CV feltöltés folyamat:**

1. Ellenőrzi, hogy van-e már CV. Ha igen, hiba. 2. A fájlnev biztonságos normalizálása. 3. A fájl buffer feltöltése a resumes Supabase bucket-be, a felhasználó ID-val megegyező mappába.

- **CV törlés folyamat:**

1. Felhasználó létezésének ellenőrzése. 2. Fájl törlése a Storageból a mappa azonosításával.

- **CV lekérés (User Resume URL) folyamat:**

1. Ellenőrzi a CV meglétét. 2. A Supabase createSignedUrl hívásával ideiglenes (lejáratlall rendelkező) linket generál a fájl letöltéséhez.

- **Statisztika lekérés folyamata:**

1. A get_user_dashboard_stats RPC függvény hívása, amely begyűjti az értesítések, kedvencek és látogatók számát aggregálva.

- **Kedvenc hozzáadása folyamat:**

1. Megvizsgálja, hogy létezik-e a hirdetés és még nem kedvenc-e. 2. Beszúr egy új rekordot a favorites táblába (user_id, advertisement_id) párossal.

- **Kedvencek lekérése folyamat:**

1. Csatlakozik a favorites és advertisement táblákhoz a felhasználó azonosítója alapján. 2. Formázott listával tér vissza.

- **Kedvenc törlése folyamat:**

1. A hirdetés ID és User ID alapján megkeresi és törli a vonatkozó bejegyzést a favorites táblából.

- **Profil megtekintés növelése folyamat:**

1. Más (például egy cég) hívja meg. 2. A increment_profile_views automatikusan eggyel lépteti a nézettség (view_count) mezőt a statisztika táblában.

Cég Modul (companyRoute.ts / companyService.ts)

A munkáltatói cégek profiljainak es statisztikáinak lekérése, és az újonnan felvett alkalmazottak (diákok) kezelése.

Végpont	Függvény	Összefoglalás
GET /Api/companies/me	getCompanyProfile	Lekéri az adott cég nyilvános profilját és adatait.
PATCH /Api/companies/me	updateCompanyProfile	Szerkeszti az aktuális cég bemutatkozását és címadatait.
GET /Api/companies/me/stats	getCompanyStats	Statisztikát (RPC) és a 4 legfrissebb jelentkezést állítja össze.
GET /Api/companies/me/employees	getEmployees	Visszaadja a felvett, céghez tartozó diákok (munkavállalók) listáját.
DELETE /Api/companies/me/employees/:id	deleteEmployee	Töröl egy aktív alkalmazottat a cégtől, és automatikusan küld is neki erről egy rendszerüzenetet (értesítést).

- **Profil lekérés folyamata:**
1. A companyId kinyerése a tokenből. 2. A companies tábla megfelelő sorának lekérése.
- **Profil frissítés folyamata:**
1. A companyId kinyerése a tokenből. 2. A companies tábla megfelelő sorának módosítása a kapott adatokkal.
- **Statisztika lekérés folyamata:**
1. get_company_dashboard_stats RPC hívása a fő KPI-ok (megtekintés, jelentkezők) eléréséhez.

- **Alkalmazottak lekérése folyamata:**
1. Az employees táblából az aktuális munkavállalók lekérése joinolva a user adatokkal.
- **Alkalmazott törlése folyamata:**
1. Törlés esetén az alkalmazott kikerül az employees táblából. 2. Rendszerüzenetet generál a törölt alkalmazott számára.

Hirdetések (advertisementRoute.ts / advertisementService.ts)

Az álláshirdetések életciklusának menedzselése.

Végpont	Függvény	Összefoglalás
POST /Api/advertisements	createAdvertisement	Új álláshirdetést hoz létre a limitszabályozás ellenőrzése (max 3/cég) után.
GET /Api/advertisements	getAllAdvertisements	Lekéri az összes, jelenleg aktív státuszú álláshirdetést.
GET /Api/advertisements/:id	getAdvertisementById	Egy konkrét álláshirdetés teljes részletes lekérése, beleértve a cégnevet is.
GET /Api/companies/me/advertisements	getCompanyAdvertisements	Lekéri a munkáltató (cég) által feladott saját hirdetéseket.
PATCH /Api/advertisements/:id	updateAdvertisement	Módosítja egy állás paramétereit (leírás, órabér stb.).
PATCH /Api/advertisements/:id/status	updateAdvertisementStatus	Be-/Kikapcsolja egy hirdetés

		aktív/inaktív állapotát.
GET /Api/advertisements/search	searchAdvertisements	Teljes szöveges keresést, bér- és városszűrést valósít meg lapozással.
GET /Api/advertisements/top3	getTopAdvertisements	Visszaadja a 3 legfrissebb aktív állást (főoldalra).
PATCH /Api/advertisements/:id/views	incrementClickCount	Az (RPC) automatikusan inkrementálja a hirdetés nézettségét (kattintás).

- **Hirdetés létrehozása folyamata:**
 1. Validációk (kötelező mezők, negatív bér kiszűrése).
 2. Hirdetési limit ellenőrzése (max 3 aktív).
 3. Beszúrás az advertisement táblába.
- **Összes hirdetés lekérése folyamata:**
 1. Lekéri az összes aktív státuszú hirdetést csatolva a cégalapokat (név, logo) a főoldali listához.
- **Egy hirdetés lekérése folyamata:**
 1. Adott azonosítójú hirdetés részletes adatainak lekérése az advertisement táblából a cégalapokkal joinolva.
- **Saját hirdetések lekérése folyamata:**
 1. A hitelesített cég összes (aktív és inaktív) hirdetésének lekérdezése az adminisztrációhoz.
- **Hirdetés frissítése folyamata:**
 1. A bejövő módosítások validálása.
 2. Meglévő hirdetés adatainak felülírása az adatbázisban.
- **Hirdetés státusz frissítése folyamata:**
 1. Átkapcsolja az aktív/inaktív (true/false) jelzőt, így levéve vagy visszatéve a hirdetést a publikus listákba.

- **Keresés folyamata:**
 1. A searchAdvertisements összetett lekérdezést épít fel (Title keresés ILIKE-al, bérminimum, lokációsűrés). 2. Pagination (lapozás) alkalmazása.
- **Top 3 hirdetés lekérése folyamata:**
 1. Visszaadja a 3 legfrissebb aktív állást limitálva, a főoldali kiemeléshez.
- **Megtekintés (View) növelés folyamata:**
 1. Kattintás esetén az increment_click_count RPC fut le, atomi módon növelve a nézettséget.

Jelentkezések (applicantRoute.ts / applicantService.ts)

A diákok jelentkezési folyamata és a cégek felvételi/elutasítási logikája.

Végpont	Függvény	Összefoglalás
POST /Api/applications	submitApplication	A felhasználó jelentkezik egy aktív hirdetésre; rekord készül a job_applications-ben.
GET /Api/users/me/applications	getUserApplications	Lekéri a diák leadott jelentkezéseit (státuszokkal) és az aktív munkakapcsolatait (alkalmazott).
GET /Api/advertisements/:id/applications	getApplicantsForAdvertisement	Lekéri egy állás összes beküldött jelentkezőjét a cég számára, diák adatokkal és önéletrajzuk elérhetőségével.
PATCH /Api/applications/:id/status	acceptApplication/rejectApplication	Elutasítja (rejected) vagy felveszi (accepted) a

jelentkezőt az állásra.

GET /Api/applications/:id/resume

getApplicantResumeUrl

A cég letöltheti a diák CV-jét (jogosultság-ellenőrzéssel egybekötve). Itt kerül felszámolásra egy Resume-view a diáknak.

- **Jelentkezés (Submit) folyamat:**
 1. Új rekord beszúrása a job_applications táblába submitted státusszal. 2. Megakadályozza a duplikált jelentkezést.
- **Felhasználó jelentkezéseinek lekérése folyamat:**
 1. Lekéri a diák összes beadott jelentkezését és azok státuszát, joinolva a hirdetés- és cégs adatokkal.
- **Hirdetés jelentkezőiknek lekérése folyamat:**
 1. Lekéri az adott hirdetésre érkezett összes jelentkezést a diákprofilokkal együtt a cég felülete számára.
- **Elfogadás (Accept) folyamat:**
 1. Ellenőrzi a cég jogosultságát. 2. A employees táblában (alkalmazottként) regisztrálja a diákot. 3. A jelentkezés státuszát accepted-re állítja. 4. Értesítő rendszerüzenetet generál.
- **Elutasítás (Reject) folyamat:**
 1. A jelentkezés státuszát rejected-re állítja alkalmazott generálás nélkül. 2. Rendszerüzenetet küld az elutasításról a diáknak.
- **Jelentkező önéletrajzának lekérése folyamat:**
 1. Ellenőrzi a cég jogosultságát a konkrét jelentkező felett. 2. Ideiglenes linket generál a diák CV-jéhez a Storage-ből. 3. Felszámol egy Resume-view megtekintést a felhasználó javára.

Kommunikáció (messageRoute.ts / messageService.ts)

A privát chat üzenetek és chatpartnerek (conversations) adatbázis műveletei.

Végpont

Függvény

Összefoglalás

GET /Api/messages/conversations (User/Company)	getUserChatPartners / getCompany...	Lekérdezi az aktív beszélgetéseket és kivel van levelezése az adott fióknak.
GET /Api/messages/conversations/:id	getChatMessages	Betölti egy adott cég/diák páros chatelőzményét (és a bejövő üzeneteket is_read=true-ra állítja).
POST /Api/messages	sendMessage	Új chat üzenetet szúr be a messages táblába. (Ezután a RealtimeServer kiküldi WebSocketeken).

- **Beszélgetőpartnerek lekérése:**
1. Megállapítja a hívó típusát (Cég vagy User). 2. RPC hívással (get_user_chat_partners vagy get_company_chat_partners) kigyűjtik az egyedi beszélgetőpartnereket az utolsó üzenettel és időbélyeggel.
- **Chat előzmények lekérése:**
1. Betölti az adott partnerek közötti korábbi üzenetváltásokat lapozható formában. 2. Ezzel egy időben a rá vonatkozó olvasatlan (is_read=false) üzenetek státuszát true-ra frissíti az adatbázisban.
- **Üzenetküldés folyamata:**
1. A messages táblába egy új üzenet rekord beszúrása (ki küldte, kinek, tartalom, típus). 2. A háttérben futó RealtimeServer WebSocketten keresztül ("Broadcast") azonnal továbbítja az élő kliensek felé.

Rendszerüzenetek (systemmessageRoute.ts / systemmessageService.ts)

Automatizált adminisztratív vagy eseménynative értesítések kezelése.

Végpont	Függvény	Összefoglalás
GET /Api/notifications	getUserSystemMessages	Lekéri az el nem olvasott értesítéseket.
PATCH /Api/notifications/:id/read	markSystemMessageAsRead	Olvasottnak jelöl egy rendszerüzenetet (beszúr

egy `system_message_reads` sort).

- **Értesítések listázása folyamat:**

1. Lekéri azokat az üzeneteket a `system_messages` táblából, amik a célnak (`USER / COMPANY / ALL`) szólnak. 2. Filtereléssel (a `system_message_reads` ellenőrzésével) csak az olvasatlan értesítéseket adja vissza.

- **Értesítés olvasottnak jelölése folyamat:**

1. A `markSystemMessageAsRead` az adott üzenet ID-ját és a kliens ID-ját felhasználva beszúr egy rekordot a `system_message_reads` pivot táblába, eltüntetve azt a listából.

Email Hitelesítés & Jelszó (`emailRoute.ts` / `emailService.ts`)

A fiók aktiválási folyamatok és elfelejtett jelszavas emailek a Nodemailer használatával.

Végpont	Függvény	Összefoglalás
POST <code>/Api/auth/.../verify-email</code>	<code>send...ActivationEmail</code>	Ideiglenes JWT aktiváló token és URL-t generál, és elküldi Nodemailerrel.
PATCH <code>/Api/auth/.../verify-email</code>	<code>activate...Account</code>	A token validációja után a felhasználó <code>activated=true</code> állapotba kerül.
POST <code>/Api/auth/.../forgot-password</code>	<code>send...PasswordResetEmail</code>	Ideiglenes jelszóvisszaállító token(URL) küldése.
PATCH <code>/Api/auth/.../reset-password</code>	<code>saveNew...Password</code>	Hasheli és elmenti az új jelszót, majd invalidálja a token.

- **Hitelesítő Email küldés folyamata:**

1. Generál egy lejáratral rendelkező JWT token. 2. Eltárolja az adatbázisban (`user_email_tokens` / `company_email_tokens`) az UPSERT elv alapján. 3. Nodemailer-en (Gmail SMTP) keresztül elküldi az aktiváló linket.

- **Fiók aktiválás folyamata:**

1. A kliens végpontja ellenőrzi a token az URL-ből. 2. Validáció után módosítja az

adatbázist (activated=true állapotba léptetve a fiókot). 3. Törli az egyszer használatos token.

- **Jelszóvisszaállítási Email küldés folyamata:**

1. A megadott email alapján token generálása és adatbázisba mentése. 2. Biztonságos visszaállító link kiküldése Nodemailerrel.

- **Új jelszó beállítása folyamat:**

1. A visszaállító token és az új jelszó validálása. 2. Az új jelszó hashelése bcrypt segítségével. 3. A password_hash frissítése az adatbázisban. 4. A felhasznált token törlése.

OrionAI (OrionAIRoute.ts / OrionAI.py)

Az NLP alapú állásajánló modul a Python/FastAPI mikroszolgáltatásból és a Node Router hídból épül fel.

Végpont	Függvény	Összefoglalás
POST /Api/OrionAI	OrionAI.py API hívás / getAdvertisementsByIds	Továbbítja a kliens "mit dolgozna" szövegét a Python API-nak, majd lekéri a pontszámítás alapján visszakapott Top 10 azonosítót az adatbázisból.

- **Működési mechanizmus (OrionAI.py):**

1. Elindít egy önálló API-t lokálisan (8000). 2. Közvetlen lekéri az összes aktív hirdetést a Supabase DB-ből. 3. Szűri a kötőszavakat, lemmatizálja a bemeneti magyar szöveget, leválasztja a lokációkat. 4. paraphrase-multilingual-MiniLM-L12-v2 NLP modellel szemantikus (cosine) similaritást és pontszámot kalkulál. 5. Súlyozás: Szöveges egyezés (65%), Bér (20%), Helyszín (15%). 6. Visszaadja az optimális lista azonosítóit.